



ЧИТ-ЛИСТ QA: ПОДГОТОВКА К РЕЛИЗУ И РЕЛИЗ

Когда использовать:
при подключении
к новому проекту

Шпаргалка QA: Подготовка и выпуск релиза

Полный гид по управлению релизами для QA-инженера.

Эта презентация — **практическое руководство** для QA-специалистов. Собраны **критические чек-листы, схемы и лучшие практики** для управления качеством: от критериев готовности задачи до выкатки на боевое окружение. Инструмент для быстрого включения в новый проект и минимизации рисков.

💡 **Совет:** Сохрани эту презентацию в закладки — она пригодится при каждом новом проекте!



DoR/DoD



Gitflow



Перресс



UAT



Hotfix



Отчётность

11 блоков

Критических чек-листов для QA

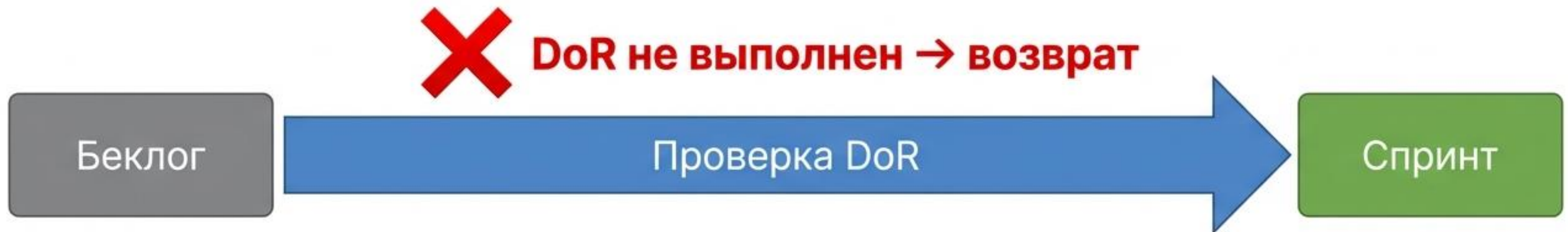
13 вопросов

Для РМ в первый день на проекте

Definition of Ready (DoR): Готовность задачи к работе

DoR — контракт между командой и PM/аналитиком. Гарантирует, что задача содержит всю информацию для старта. Без DoR команда теряет время, а продукт получает некачественные решения.

- ✓ Требования описаны
- ✓ Критерии приёмки прописаны
- ✓ Макеты приложены
- ✓ Спецификация API согласована
- ✓ Зависимости закрыты
- ✓ Задача оценена и в спринте

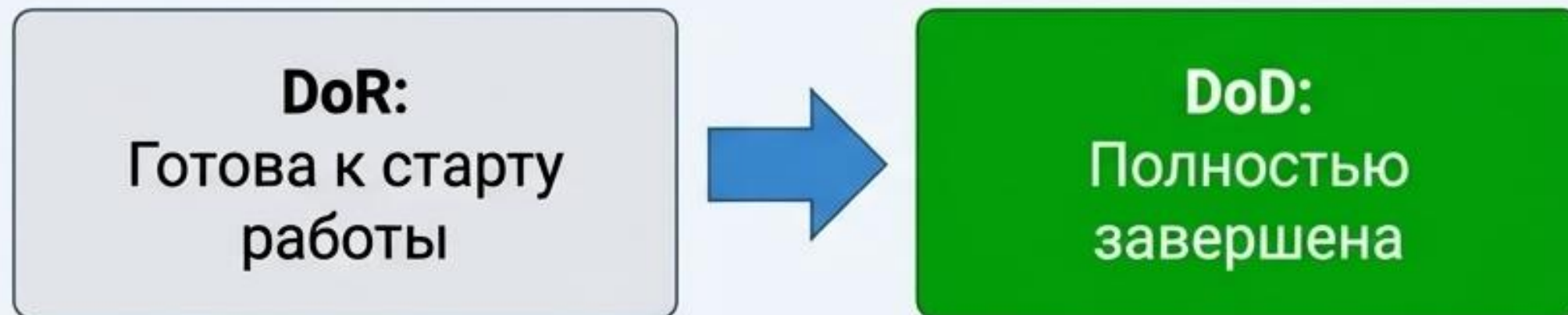


Красный флаг: DoR не зафиксирован письменно — это риск! Требуй документирования.

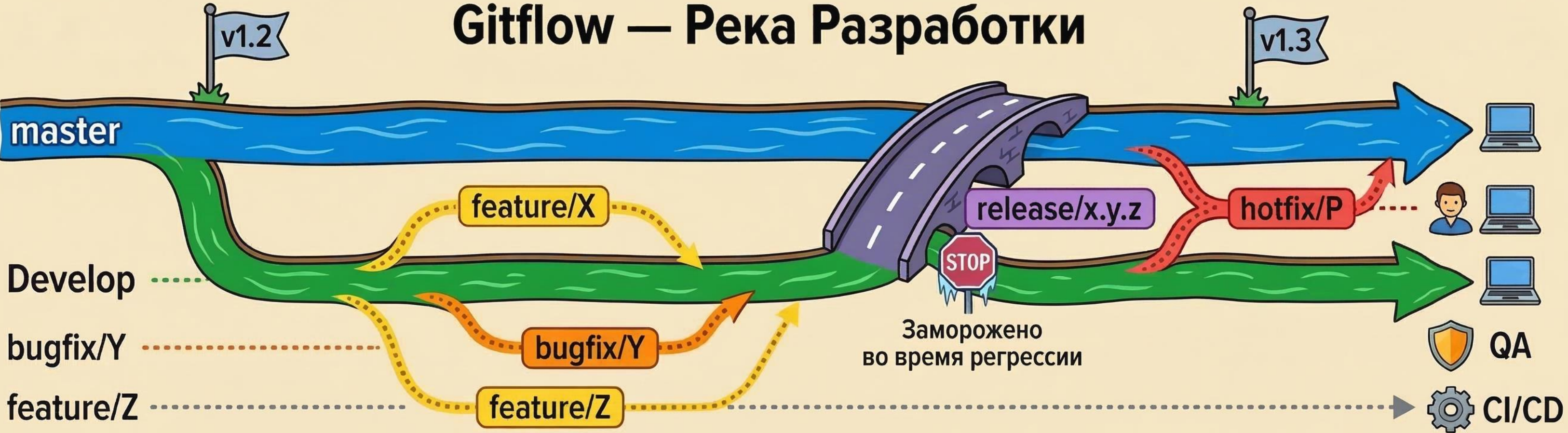
Definition of Done (DoD): Критерии завершения задачи

DoD — финальный чек-лист перед закрытием задачи. Защищает от ситуаций "вроде работает, но не протестировано". QA — последняя инстанция перед переводом в статус "Готово". Никаких "потом протестируем".

💡 **Совет:** DoD — это не формальность, а страховка от технического долга!



Gitflow — Река Разработки



Этап	Кто	Действие
1	Разработчик	Разработчик создает запрос на слияние (MR)
2	Техлид	Техлид проводит код-ревью
3	QA	Тестировщик контролирует протестированные задачи
4	CI/CD	CI/CD собирает сборку



Важно: Релизная ветка замораживается при старте регрессии — новые фичи в нее не вливаются!

Gitflow: Схема веток и рабочий процесс

Стратегия ветвления для минимизации конфликтов.
У каждой ветки чёткая роль.
Критическое правило: ветка `release` замораживается при старте регресса — никаких новых фич!



master	Продакшн-версия		Стабильные релизы
develop	Интеграция	Все фичи сливаются сюда	
feature/X	Новая функциональность		От develop
release/x.y.z	Сборка для регресса		ЗАМОРОЗКА
hotfix/P	Срочное исправление		От master

⚠ Важно: Ветка `release` ЗАМОРАЖИВАЕТСЯ при старте регресса — новые фичи НЕ вливаются!

Контроль выливки в develop

Выливка в develop — критический процесс. Каждый шаг имеет владельца. QA-роль ключевая: контролировать, что в develop попадают ТОЛЬКО протестированные задачи.



💡 **QA-роль:** Ты — последняя линия защиты перед попаданием кода в develop!

 **Красный флаг:** Разработчик сливает без подтверждения QA

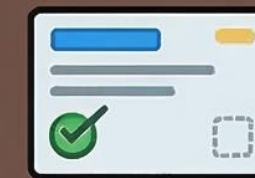
Последствие: Баги попадают в develop

Действие: Останови процесс, согласуй workflow с командой!

Критерии завершения спринта



Все задачи имеют статус
Завершено согласно DoD



Критические дефекты
спринта устранены



Демо с заказчиком проведено



Подготовлены примечания к выпуску



Бэклог обновлен



Подготовка
к выпуску

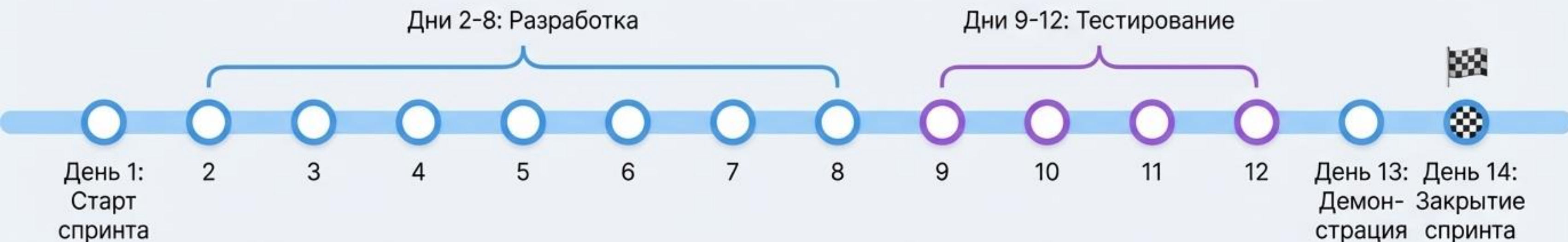
Критерии завершения спринта

Спринт не закрыт, пока не выполнены формальные критерии. Это не просто "задачи готовы", это согласованный с заказчиком результат.

Частая ошибка: закрытие без демо.

- ✓ Все задачи в статусе "Готово"
- ✓ Дефекты обработаны
- ✓ Демонстрация проведена
- ✓ Заметки к релизу готовы
- ✓ Беклог обновлён

⚠ **Красный флаг:** Спринт закрыт без демонстрации или без обратной связи от заказчика — это путь к переделкам!



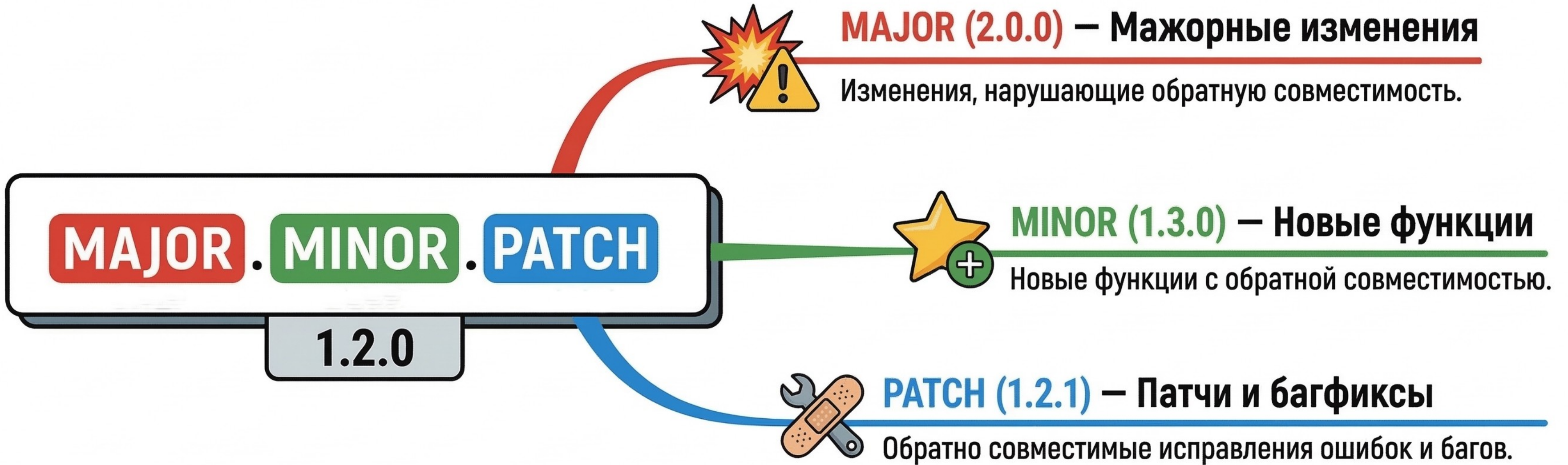
Версионирование: Анатомия SemVer

SemVer (Семантическое версионирование) — стандарт, где каждая цифра имеет строгий смысл.
QA обязан знать, где ставится тег и какая версия развёрнута на каждом окружении.

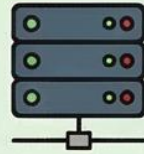
💡 **QA-задача:** Фиксируй версию на каждом окружении в Confluence — это помогает при отладке!



Dev	Test	Demo	UAT	Prod



Входные критерии



Среда
выполнения
доступна



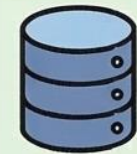
Релизная
сборка
собрана



Список
задач
получен



Тестовая документация актуальна



Тестовые
данные
подготовлены

Процесс регрессии



Пройден



Провален



Заблоки-
рован



Пропущен



Пропущен



Не запу-
щен

Когда начинается регрессия: задачи объединены в develop, создана и заморожена релизная ветка.

Выходные критерии



Нет блокирующих дефектов



Все проблемы
описаны в отчете



QA дали
рекомендацию

При завершении:
выходные критерии
выполнены.



Входные критерии Регресса: Code Freeze

Регресс можно стартовать только при выполнении критериев.

Критическая ошибка: начинать на нестабильной сборке.

Ветка release замораживается — иначе результаты регресса становятся бесполезными.

- ✓ Задачи в develop
- ✓ Релизная ветка создана
- ✓ Ветка заморожена
- ✓ Окружение доступно
- ✓ Сборка развёрнута
- ✓ Список изменений готов

ЗАМОРОЗКА: Новые фичи СТОП

Все фичи в develop → Создание релизной ветки



⚠ **Красный флаг:** Регресс стартовал БЕЗ заморозки — это гарантия хаоса и нестабильных результатов!

Старт регресса
(Новые фичи строго запрещены!)

Статусы тест-кейсов: Метрики здоровья

Выходные критерии регресса: нет блокирующих/критических дефектов, проблемы задокументированы, QA дал чёткую рекомендацию к релизу.

Причины пропусков должны быть обоснованы.

💡 **Совет:** Всегда документируй причину пропуска — это критично для оценки рисков релиз-менеджером!

- ✅ Пройден (Зелёный) — Успешно
- ❌ Упал (Красный) — Найден дефект
- 🚫 Заблокирован (Оранжевый) — Блокируется другим багом
- ⏸ Пропущен (Серый) — Требуется обоснования
- ⬜ Не выполнен (Светло-серый) — В очереди



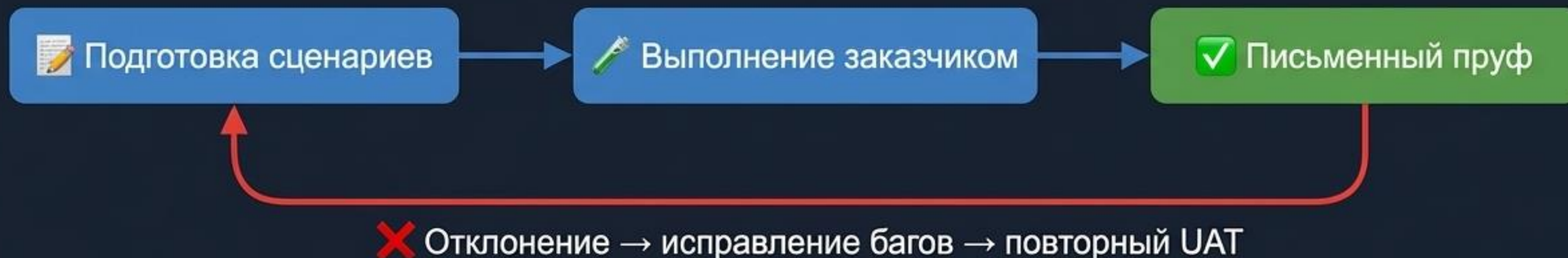
UAT: Приёмочное тестирование

Финальная проверка перед продакшеном.
Выполняется заказчиком.

Цель: подтвердить соответствие бизнес-требованиям.

Сценарии UAT согласуются ДО старта.

SLA параметры	
Окружение	UAT/Demo (НЕ продакшен!)
Участники	Представители заказчика
Формат	Заранее согласованные сценарии
Итог	Письменное подтверждение приёмки



⚠️ Красный флаг: Заказчик проводит UAT напрямую на продакшене — критический риск!

Pre-Production Alarm Zone

Typical Risks					
Risk					
	Прод ≠ Тест Конфиг	Нет плана отката	Нет бэкапа БД	Нет мониторинга	Множество команд в проне
Mitigation	Записать разницу конфигураций	Подготовить сценарий отката	DevOps создает бэкап	Настроить алерты	Согласовать окно деплоя



Риски продакшн-выкаты и митигация

Выкатка на продакшен — самый рискованный этап. К каждому риску должен быть заготовлен протокол противодействия.

Угроза	Защита
⚠️ [Угроза] Конфигурация продакшена отличается	→ 🛡️ [Защита] Зафиксировать различия до релиза
⚠️ [Угроза] Нет плана отката	→ 🛡️ [Защита] Подготовить сценарий отката
⚠️ [Угроза] Нет резервной копии БД	→ 🛡️ [Защита] DevOps делает бэкап
⚠️ [Угроза] Нет мониторинга	→ 🛡️ [Защита] Настроить алерты заранее
⚠️ [Угроза] Несколько команд деплоят	→ 🛡️ [Защита] Согласовать окно развёртывания

✅ Бэкапы БД

✅ План отката

✅ Мониторинг

✅ Окно деплоя

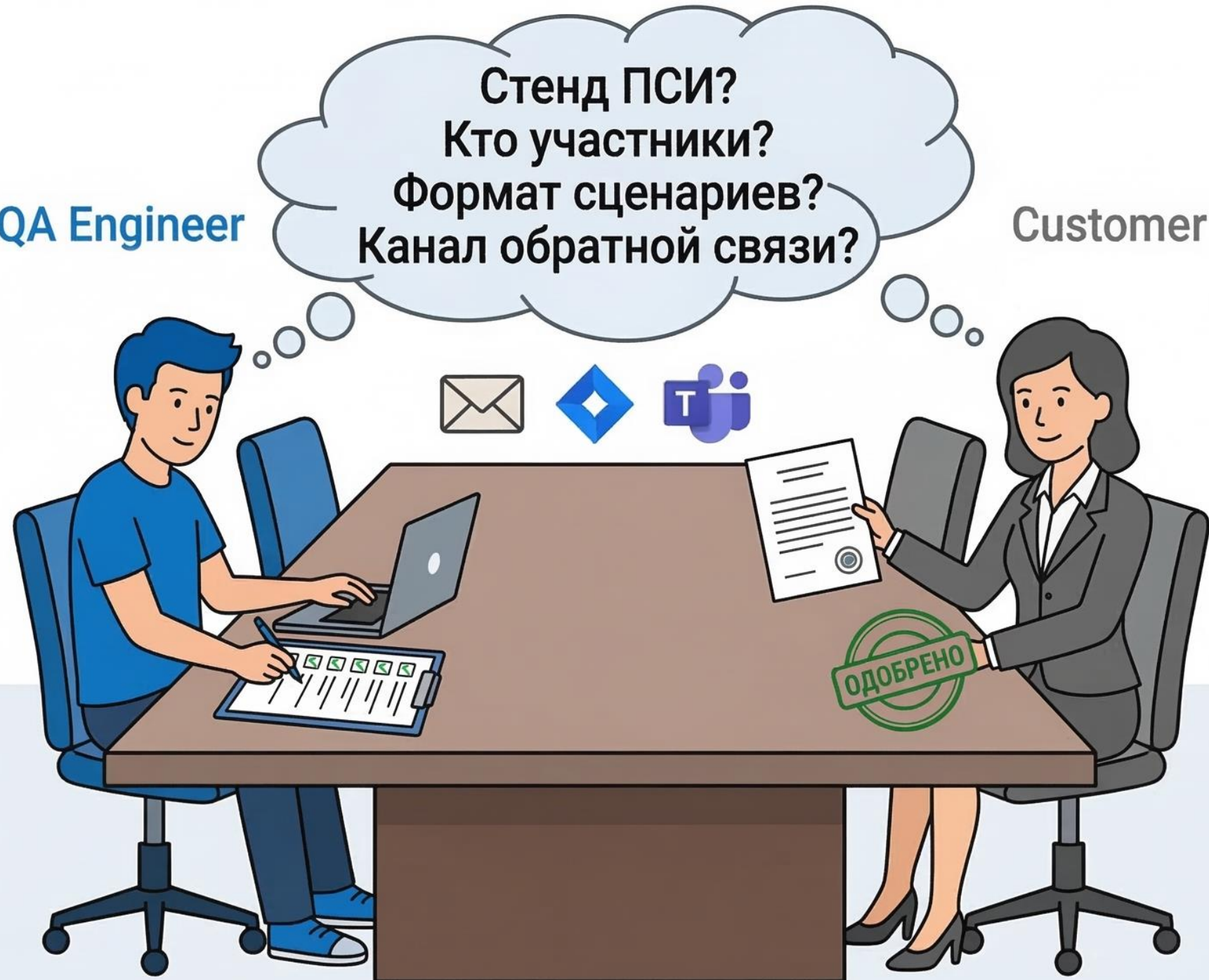
💡 **QA-роль:** Перед деплоем пройди по матрице рисков — это спасёт от катастроф.

Приемка Заказчиком

Параметров

- 1) Стенд (ПСИ/Демо)
- 2) Участники (Заказчик)
- 3) Формат сценариев (Ваши/Заказчика)
- 4) Канал ОС (Email/Jira/Teams)
- 5) SLA ответов
- 6) Критерии успешной приемки (Письменное подтверждение)

QA Engineer



Правила UAT

- 1) Сценарии согласованы
- 2) Дефекты занесены в таск-трекер
- 3) Зафиксированы проблемы: blocker/не blocker
- 4) Результат - письменная приемка/отказ



Красный флаг: Заказчик проводит ПСИ прямо на продуктив - критический риск!

Чек-листы перед развёртыванием: Зоны ответственности

Развёртывание требует координации трёх сторон. Невыполнение любого пункта создаёт риск отката релиза. QA контролирует готовность всех смежных отделов.



Разработчики

- ☐ Миграции БД
- ☐ Фич-флаги
- ☐ План отката



DevOps

- ☐ Резервные копии (БД/Конфиги)
- ☐ Пайплайн готов
- ☐ Мониторинг включен
- ☐ Окно релиза согласовано



Заказчик

- ☐ UAT завершён
- ☐ Пользователи уведомлены
- ☐ Бизнес-процессы готовы

⚠️ РАЗВЁРТЫВАНИЕ НА ПРОДАКШЕН ТОЛЬКО ПОСЛЕ ВЫПОЛНЕНИЯ ВСЕХ 3 ЧЕК-ЛИСТОВ!

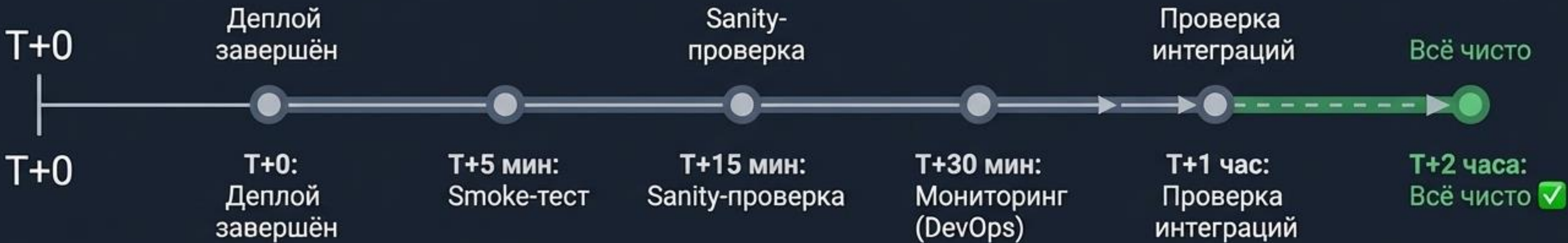
После деплоя на ПРОД



Пост-релизное тестирование: Первые часы жизни релиза

Развёртывание — это не финиш, а старт интенсивного мониторинга. QA и DevOps работают в связке для моментального обнаружения сбоев.

Smoke (QA)	Доступность, авторизация, ключевые сценарии
Sanity (QA)	Конкретные изменения данного релиза
Мониторинг (DevOps)	Логи, ошибки 5xx, время отклика
Интеграции (QA+Dev)	Платёжки, внешние API, SSO



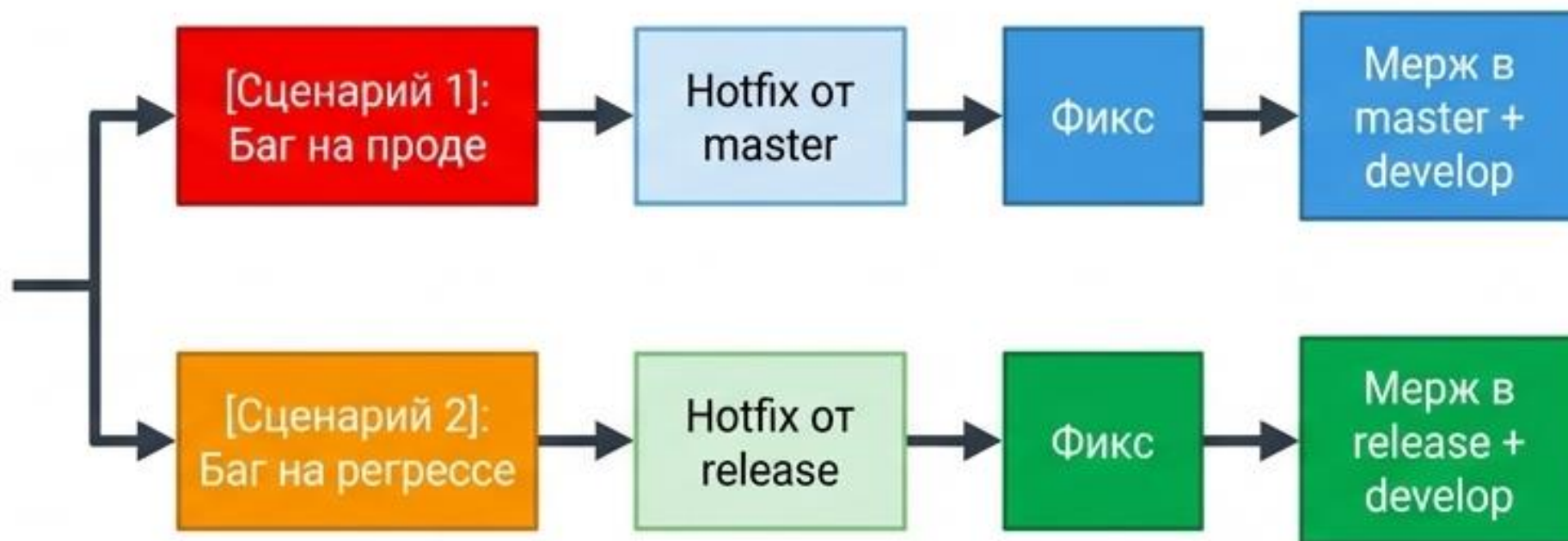
Критично: Первые 2 часа после развёртывания — самые важные для спасения релиза!

Hotfix: Экстренная хирургия на продакшене

Экстренное исправление критического бага.

Создаётся от ветки master (прод) или release (регресс).

Должен соблюдаться строгий алгоритм, чтобы фикс не породил новые баги.



⚠ Красный флаг: Hotfix сливается БЕЗ тестирования (из-за спешки) — останови процесс!

- ✓ Smoke-тест ключевых сценариев
- ✓ Целевая проверка исправления
- ✓ Регрессия затронутой области
- ✓ Проверка на prod-подобном окружении
- ✓ Версия обновлена (PATCH +1)
- ✓ Заметки к релизу готовы



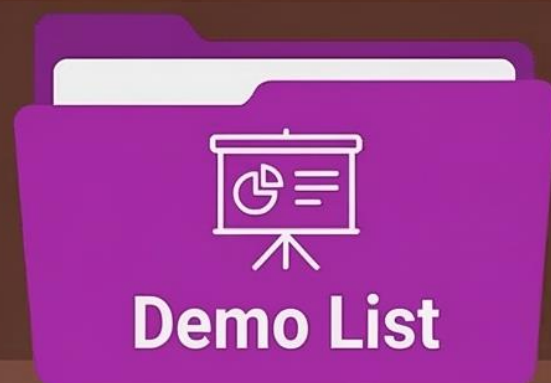
— Новые функции, исправления,
известные проблемы, миграции
Кто: QA / Аналитик / Разработчик
Где: Confluence / Wiki



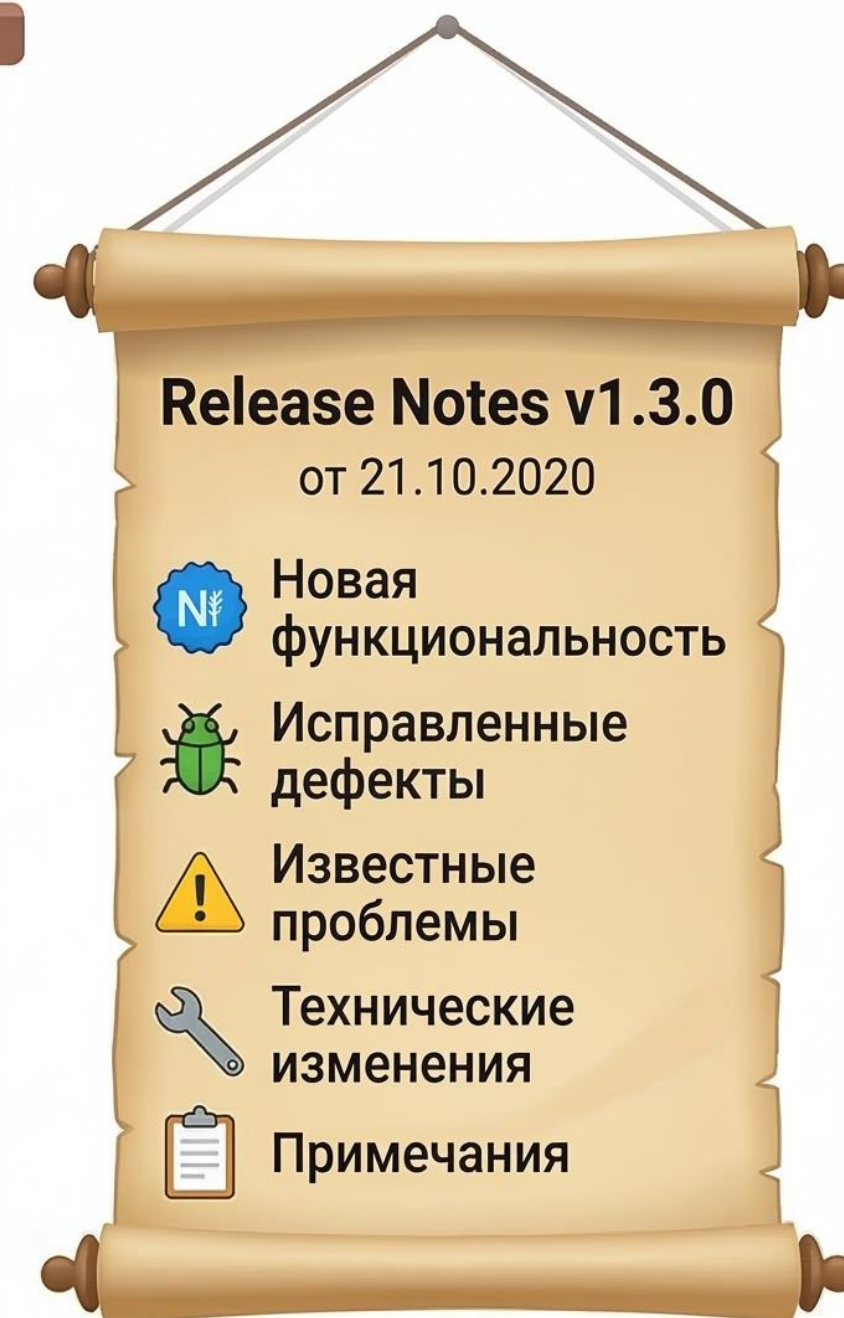
— Область тестирования, результаты,
дефекты, рекомендации
Кто: QA
Где: Confluence / Jira



— Инструкции для пользователей
по новой функциональности
Кто: Аналитик / QA
Где: Confluence



— Что было показано заказчику на демо
Кто: ПМ / QA
Где: Почта / Confluence



Набор выживания QA: Вопросы РМ и Красные флаги

Задай эти вопросы тимлиду в первый день. Это сэкономит месяцы недопонимания.
Если видишь "Красный флаг" — твоя задача как QA немедленно эскалировать проблему.

💬 1. Где описан рабочий процесс?	💬 2. Что считается DoR и DoD?
💬 3. Кто аппрувит слияние в develop?	💬 4. Как собирается релизная сборка?
💬 5. Какая модель ветвления?	💬 6. Кто назначает номер версии?
💬 7. Чем отличается Prod от UAT?	💬 8. Кто проводит UAT от заказчика?
💬 9. Какие риски при выкатке на Prod?	💬 10. Кто делает деплой? выкатки?
💬 12. Какие артефакты после регресса?	💬 13. Какой SLA на Hotfix?



Нет письменного DoR/DoD



Конфигурации продакшена не зафиксированы



Нет плана отката



Заказчик принимает на продакшене без UAT



Релизная ветка не замораживается



Действие: Сохрани эти вопросы. Это твой главный инструмент для наведения порядка на любом проекте.



Отчет по тестированию – v1.3.0 Регресс

Основное

⚙	Версия/Сборка/Тег	v1.3.0
💻	Тестовый стенд	Стенд
📅	Период тестирования	10.09 – 2021
👥	Команда QA	Командата QA

Область тестирования

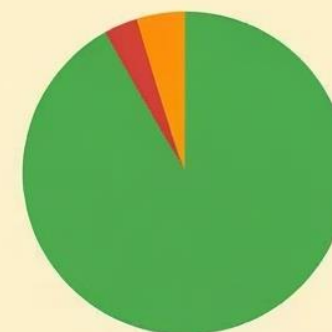
- ✅ Включено: список модулей
Модулей, модулей, акмак модулей, модулей, модулей, модулей, убизования модулей
- ❌ Исключено: что не включено

Известные ограничения/Риски

- ⚠️ Риски: своряма незвестные ограниченнов
- ⚠️ Риски: известные ограничения Риски OS
- ⚠️ Риски: дерептинватье эта риски

Результаты

- ✅ Пройдено: 142
- ❌ Не пройдено: 3
- 🚫 Заблокировано: 1
- ➡ Пропущено: 5



Критические дефекты

ID	Описание	Течность	Статус
PRJ-115	Сбой при запуске на Mac OS	Критическая	Открыто
...			

Рекомендация QA

❌ **Не рекомендуется**

Изосиникация подреставляло провесонит рекомендация, она тожиа рекомендується в негомия рекомендауции.

Гоним

Обязательный артефакт перед выпуском